

软件开发文档

程序设计实践课 · 项目开发文档

一、项目基本信息

项目	内容
项目名称	MemoDoc —— 带长期记忆的教育资料问答学习助手
作者姓名	吴本昭
学号	24375097
专业 / 班级	软件工程 / 242112
课程	程序设计实践
开发周期	2026 年 8 月 25 日 — 2026 年 9 月 5 日
技术栈	后端：Python 3.12 + FastAPI + Uvicorn + pydantic；核心：手写 BM25 + 本地向量库 + DeepSeek API + 本地 bge 模型；前端：React 18 + TypeScript 5 + Vite 5 + Ant Design 5 + Zustand 4
代码总行数	约 5000 行（Python ≈ 3300 行，TS/TSX ≈ 1200 行，CSS ≈ 500 行，不含前端依赖）
代码仓库	https://github.com/Slowswingcloud/MemoDoc.git
运行环境	Windows 10/11；Python 3.11–3.12（推荐 3.12）；Node.js 18+（仅前端构建需要）；需联网调用 DeepSeek API 并首次下载本地模型

二、项目概述

2.1 项目背景与动机

课程学习中最常见的痛点是"资料一堆、问不到点上"：学生手里有课件、作业、往年题，但复习提问时没人及时答疑，翻资料效率低；教师无法低成本地了解全班普遍的薄弱点。MemoDoc 面向这一场景，做一个**把课程资料变成"记得住你的问答助教"的本地化系统：教师上传资料形成班级知识库，学生像聊天一样提问，回答强制附带 [1][2] 式原文引用、可点击溯源，并且系统会跨会话记住用户（身份与偏好），新会话自动带上相关记忆回答。**

市面上通用大模型（ChatGPT/DeepSeek）虽有 RAG 类工具，但存在回答不可溯源、不记忆用户、数据要上传云端等问题；开源参考实现 Kotaemon（RAG UI）与 mem0（长期记忆）功能完整但面向通用场景、依赖较重。本作品的差异点在于：**面向教学闭环自研轻量全链路——手写 BM25 稀疏检索与自研向量库（零原生依**

赖、数据全本地、可审计)，记忆采用 mem0 的结构化事实范式，并用"引用核查"闭环保证回答可追溯、抗幻觉。对个人而言，本项目让我把课堂上学的软件工程方法（需求→设计→分层→测试→文档）完整走了一遍，并亲手实现了检索、重排、记忆抽取等此前只停留在"调 API"层面的技术。

2.2 项目简介

MemoDoc 是一个**带长期记忆的文档问答智能体**：一句话说，就是"把课程资料变成一位记得住你的助教"。用户上传 PDF/Markdown/文本文件后，系统自动解析分块、向量化并建立检索索引；用户以自然语言提问，系统在文件库中检索相关片段、结合对该用户的长期记忆与对话历史，流式生成带 [n] 引用的回答；点击引用编号即可打开源文件定位，回答中的每条引用还会被大模型逐条核查并打上"✓已核查/△不支持"标记。它同时提供多用户账号与管理员权限、会话管理、文档按标签归类与过滤检索等能力，既能单人自用，也能在一个班级范围内共享知识库。

三、需求分析

3.1 功能需求

编号	功能模块	功能描述	输入	输出
F01	用户认证	注册/登录/登出；区分普通用户 user 与管理 员 admin；管理员可查看全部用户	用户名、 密码、角 色	登录令牌、用户信息
F02	会话管理	新建/切换/删除会话；会话按用户隔离，历史 可回放	会话操作 指令	session_id、历史消息列 表
F03	文档上传 与索引	批量上传 PDF/MD/TXT，自动解析→分块→向 量化→建 BM25 索引；未填标签时自动打标 签	文件、可 选标签	每份文档的块数、索引 模式、标签
F04	文档库管 理	文档列表、标签增删、下载源文件、删除 （仅上传者或管理员）	文档名、 标签	文档信息、操作结果
F05	标签检索 区间	提问前勾选若干标签，把检索限定在带这些 标签的文档内	标签集合	（影响检索范围）
F06	文档问答 （流式）	结合检索片段、相关记忆与历史流式回答； 支持标签过滤	问题文本	SSE 流：来源→逐字增 量→引用核查→完成
F07	引用溯源 与核查	回答中 [n] 引用对应真实片段；点击可打开 源文件；每条引用经 LLM 核查打标	回答文本	引用 chip、✓已核查/△ 不支持 徽标
F08	长期记忆	每轮问答后自动抽取用户身份/偏好事实，去 重、冲突更新入库；新会话自动注入相关记 忆	对话内容	记忆事实（可查、可清 空）
F09	管理员用 户管理	管理员查看全部注册用户	—	用户列表
F10	命令行与 模型工具	CLI 索引/问答/查看文档与记忆/标签管理；模 型断点续传下载	子命令与 参数	控制台输出

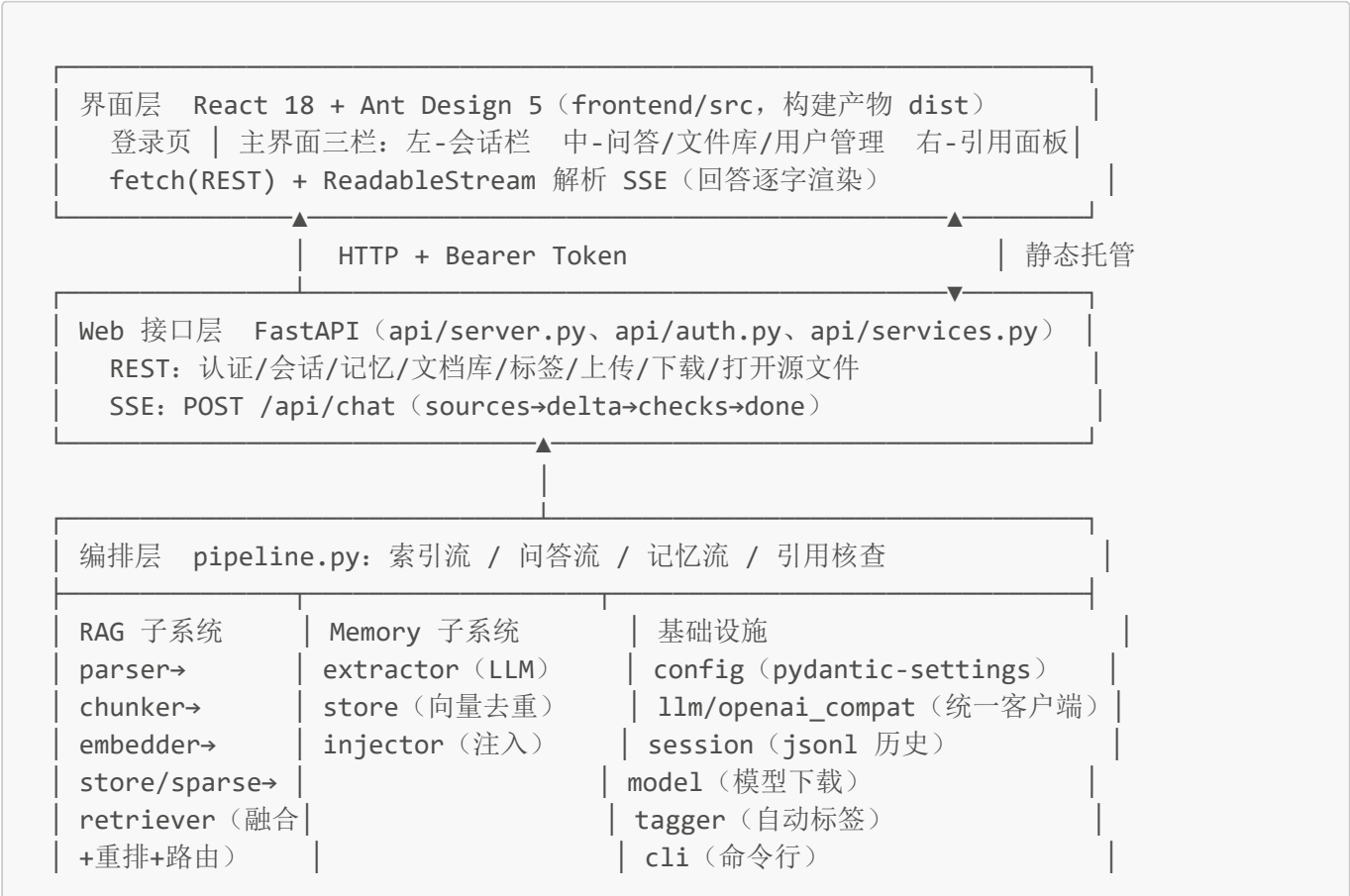
3.2 非功能需求

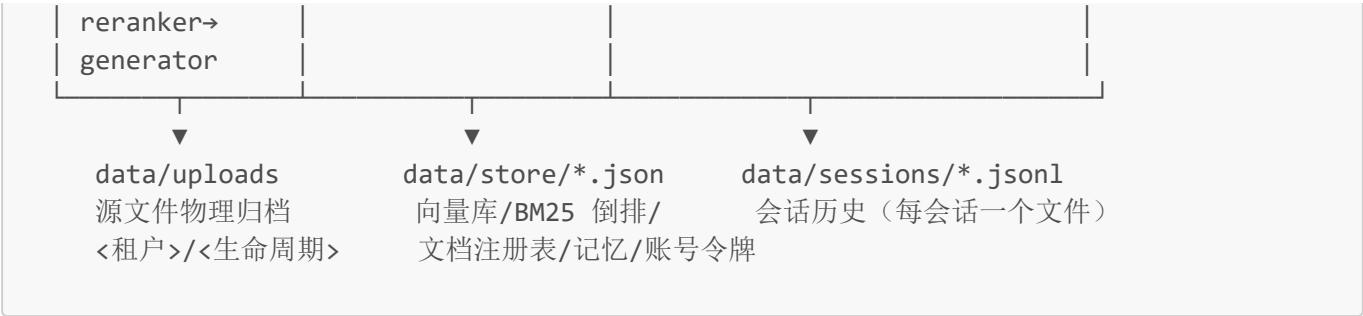
需求类别	具体要求
性能	演示规模（数百块）下检索毫秒级完成；CPU 重排 + 流式生成下首字延迟 < 30s；回答流式逐字输出不阻塞输入
易用性	提问三步内完成（登录→输入问题→发送）；提供建议问题；首启无账号时自动创建默认管理员；embedding/重排模型缺失时自动降级仍可演示
兼容性	Windows 本地一键启动；生产模式由 FastAPI 托管前端构建产物（单端口 8000）；开发模式 Vite 5173 自动代理 <code>/api</code>
安全性	密码 HMAC-SHA256 加盐哈希存储；登录令牌 7 天 TTL；接口 Bearer 鉴权；文档删除有权限控制（上传者或管理员）；"打开源文件"做路径白名单校验
数据本地性	全部数据（索引、向量、记忆、会话、账号）以 JSON/jsonl 存于本地 <code>data/</code> ，可审计、可整体迁移，数据不出校
可维护性	三层解耦：核心 <code>src/</code> 与 Web 层分离；核心模块单一职责；评测与冒烟脚本齐全（28/28 接口用例通过）

四、系统设计

4.1 整体架构

系统采用"界面层 → Web 接口层 → 编排层 → 能力子系统/基础设施 → 数据层"的分层结构，并外接两路 AI 服务（云端 DeepSeek 大模型与本地 bge 系列模型）。





AI 服务：① DeepSeek（OpenAI 兼容）负责生成、翻译、打标签、抽取事实、引用核查；② 本地 BAAI/bge-small-zh-v1.5 做文档/查询向量化、BAAI/bge-reranker-v2-m3 做候选精排（均可离线，缺失时检索自动降级为纯 BM25/关键词模式）。

4.2 模块划分

后端核心（src/memodoc/）

模块 / 文件	职责	主要接口 / 函数
config.py	pydantic-settings 集中配置 (.env 可覆盖)	settings
llm/openai_compat.py	DeepSeek/Kimi 统一 OpenAI 兼容客户端	LLMClient.chat / chat_stream / chat_json；单例 llm
rag/parser.py	PDF（PyMuPDF 块级提取）/MD/TXT → 纯文本	parse_file()
rag/chunker.py	标题感知分块：1–3 级标题切段，段内滑窗重叠；4 级+标题并入正文	chunk_text()
rag/embedder.py	本地 bge 向量化，失败自动降级	Embedder.embed()；QUERY_INSTRUCTION
rag/store.py	自研向量库（numpy 余弦 + JSON 持久化）、块存储、文档注册表	VectorStore / JsonVectorStore / DocumentRegistry
rag/sparse.py	手写 BM25（jieba/英文自适应分词，倒排索引 JSON 持久化）	BM25Index.build / search；tokenize()
rag/reranker.py	bge-reranker 交叉编码器精排（缺失自动跳过）	Reranker.rerank()
rag/retriever.py	向量+BM25 双路召回、归一化融合、重排、跨语言双语检索、文档路由、标签过滤	Retriever.retrieve()；_fuse()
rag/generator.py	拼装系统提示（记忆 + 编号片段 + 历史），流式/非流式生成，强制 [n] 引用	Generator.build_messages / stream / complete

模块 / 文件	职责	主要接口 / 函数
memory/extractor.py	每轮对话后 LLM 抽取用户身份/偏好事实	FactExtractor.extract()
memory/store.py	记忆向量库：相似去重 + 同属性冲突更新	MemoryStore.add / search / all / clear
memory/injector.py	按问题召回相关记忆并格式化成提示片段	MemoryInjector.facts / format / inject
session.py	会话历史 jsonl 持久化	SessionStore.all / recent / append / list_sessions / delete
tagger.py	文档自动打标签：LLM 建议 + 启发式兜底	suggest_tags() / fallback_tags()
pipeline.py	三条链路编排 + 引用核查 + 文档/标签/记忆管理	index / answer_stream / answer / check_citations / delete_doc / documents / all_tags / ...
model.py	HF 模型断点续传下载 (Range + 重试)	download_model() / download_reranker()
cli.py	命令行入口	memodoc index / ask / docs / memories / tag / autotag / reindex / download-model

Web 层与入口（根目录、api/）

模块 / 文件	职责	主要接口 / 函数
server.py	启动入口：uvicorn 拉起 FastAPI (8000)	uvicorn.run("api.server:app", ...)
api/server.py	FastAPI 路由：鉴权、会话、SSE 问答、文档库、标签、上传、下载、打开源文件；静态托管前端构建产物	app; register/login/...、chat() (SSE)、upload() 等
api/auth.py	本地 JSON 账号系统（角色 user/admin）	AuthService.register / login / logout / get_user / list_users / ensure_admin
api/services.py	产品层：文档所有权元数据（doc_meta.json），合并核心注册表	DocService.upsert / remove / owner_of / list

前端（frontend/src/）

模块 / 文件	职责	主要接口 / 函数
main.tsx / App.tsx	React 挂载；三栏主布局与 Tab 切换（问答/文件库/用户管理）	App()
types.ts	共享类型（用户/会话/消息/引用/核查/文档/记忆/SSE 事件）	—
api.ts	fetch 封装 + SSE 流式客户端	api.*; chatStream()
store.ts	Zustand 全局状态与动作（登录/会话/发送/文档/标签）	useStore
components/AuthPage.tsx	登录/注册页（注册可选角色）	—
components/Sidebar.tsx	左侧会话栏：新建/切换/删除会话	—
components/ChatPanel.tsx	问答主区：消息流、建议问题、标签多选过滤、输入发送	—
components/MessageItem.tsx	消息气泡：把 [n] 解析为可点击引用 chip	splitCites()
components/SourcesPanel.tsx	右栏引用来源卡片（点击打开源文件 + 核查徽标）	—
components/DocLibrary.tsx	文件库：上传（带标签）、表格、标签增删、下载、删除	—
components/AdminUsers.tsx	管理员用户列表	—
styles.css	全局样式（布局/气泡/卡片/引用 chip）	—

测试 (tests/)： `smoke.py`（冒烟）、`test_project.py`（FastAPI TestClient 接口用例，28/28 通过）、`eval.py`（三档检索召回/引用准确率）、`eval_papers.py`（英文论文跨语言评测）、`eval_ragas.py`（RAGAS 端到端指标）、`eval_softeng.py`（软件工程演示集）。

4.3 数据结构 / 存储设计

系统不依赖数据库，全部以 JSON / jsonl 文件落盘在 `data/`，便于审计、备份与整体迁移。

```
data/
├── uploads/<租户>/<生命周期>/          # 上传源文件物理归档（如
default/active/demo_doc.md）
├── store/
│   ├── docs.json          # 分块向量库：块文本 + embedding +
meta{section,source,tenant,lifecycle,tags}
│   ├── sparse.json        # BM25 倒排索引（tokenizer 版本号，变更自动重建）
│   ├── documents.json     # 文档注册表
│   ├── doc_meta.json      # 文档所有权（owner）
│   ├── memories.json      # 长期记忆向量库
│   └── users.json         # 账号（加盐哈希密码）
```

└─ tokens.json

└─ sessions/

登录令牌（含过期时间）
会话历史，每会话一个 .jsonl

文档注册表样例（documents.json）：

```
{
  "demo_doc": {
    "source": "data/uploads/default/active/demo_doc.md",
    "chunks": 24,
    "indexed_at": 1788...,
    "tenant": "default",
    "lifecycle": "active",
    "tags": ["社团", "入社条件"]
  }
}
```

记忆事实样例（memories.json，mem0 结构化事实范式）：

```
{
  "id": "mem-1a2b3c4d5e6f",
  "text": "我是大一新生，状态图画不好",
  "meta": {
    "type": "identity",
    "subject": "年级",
    "user_id": "zhangsan",
    "ts": 1788...
  }
}
```

会话历史样例（data/sessions/zhangsan_abc123.jsonl，每行一条）：

```
{"role": "user", "content": "状态图怎么画？", "ts": 1788...}
{"role": "assistant", "content": "绘制状态图一般分三步.....[1]", "ts": 1788...}
```

核心内存对象：Chunk{id, text, doc_name, section_path, meta}（分块）、Retrieved{id, index, text, doc_name, section, score, source}（检索结果，index 即回答中引用编号 [n]）。

4.4 界面设计

界面	主要元素	核心交互
登录 / 注册页	应用标识、账号密码输入、角色选择（user/admin，仅注册时）	登录 → 进入主界面；无账号先注册
主界面（问答 Tab）	顶部头部（应用名/角色徽标/用户/退出）；中间消息流与建议问题；底部标签过滤 + 输入	点建议问题或输入提问 → 流式逐字回答 → 回答中 [n] 为可点击引用

界面	主要元素	核心交互
	框	
主界面（右栏引用来源）	引用卡片（编号、文档·章节、预览、核查徽标）	点击卡片 → 系统默认程序打开源文件定位
左侧会话栏	新对话按钮、会话列表	新建 / 点击切换 / 删除会话，历史即时回放
文件库 Tab	上传区（多文件 + 标签输入）、文档表格（名称/块数/标签/所有者）	上传自动索引并自动打标签；表格内可加/删标签、下载、删除（非本人非管理员会提示 403）
用户管理 Tab（管理员专属）	全部用户列表	管理员查看各账号与注册时间

（此处可插入实际运行截图：登录页、问答页含引用面板、文件库页，共 2-3 张。）

五、关键算法与技术实现

5.1 关键技术点列表

- 混合检索：向量 + 手写 BM25 双路召回、归一化加权融合、交叉编码器精排
- 长期记忆：LLM 事实抽取 → 向量相似去重 / 同属性冲突更新 → 相关记忆注入
- 引用可信闭环：编号引用机制 + 生成后 LLM 逐条引用核查
- SSE 流式问答全栈管线：事件协议 + 前端逐字渲染

5.2 技术点详解

技术点 ①：混合检索（向量 + BM25 融合 + 精排）

问题背景：纯向量检索对专有名词、精确数字不敏感，纯关键词（BM25）又无法处理同义改写；单一检索召回不足会直接导致回答漏项。需要把两类互补信号融合，再用精排模型把候选压到可放进提示词的数量。

算法思路：

输入：查询 query、候选数 n、融合权重 w
输出：排序后的 top_k 片段

1. dense = 向量召回(query, n) # bge 编码 + numpy 余弦取前 n

2. sparse = BM25 检索(query, n) # jieba/英文自适应分词打分取前 n

3. 若两条通道都有结果：
对 dense、sparse 分数各自 min-max 归一化到 [0,1]
final(id) = w·dense(id) + (1-w)·sparse(id)，按 id 合并去重

4. 用交叉编码器对 (query, 候选片段) 逐对打分，取精排后的 top_k

5. 输出并按 1..k 编号 — 编号即回答中的 [n]

扩展：中文查询先 LLM 翻译成英文 → 双语各自召回按 id 合并（跨语言）；
查询命中某文档标题 → 先域内检索，落空自动回退全局（文档路由）；
可传标签集合把检索限定在带这些标签的块内（标签过滤）。

关键代码 (src/memodoc/rag/retriever.py 的 _fuse):

```
def _fuse(dense, sparse, w):
    """加权融合: 各自 min-max 归一化到 [0,1], 再 final = w*dense + (1-w)*sparse."""
    d_min = min(r.score for r in dense); d_max = max(r.score for r in dense)
    s_min = min(r.score for r in sparse); s_max = max(r.score for r in sparse)

    def norm(v, lo, hi):
        return (v - lo) / (hi - lo) if hi > lo else 1.0

    merged = {}
    for r in dense:
        merged[r.id] = Retrieved(id=r.id, text=r.text, doc_name=r.doc_name,
                                section=r.section, source=r.source,
                                score=w * norm(r.score, d_min, d_max))

    for r in sparse:
        if r.id in merged:
            merged[r.id].score += (1 - w) * norm(r.score, s_min, s_max)
        else:
            merged[r.id] = Retrieved(id=r.id, text=r.text, doc_name=r.doc_name,
                                    section=r.section, source=r.source,
                                    score=(1 - w) * norm(r.score, s_min, s_max))

    return sorted(merged.values(), key=lambda x: -x.score)
```

效果 / 说明: 评测脚本中对比纯向量 / 纯 BM25 / 融合三档召回, 融合档在关键词精确命中与语义近义上均不塌陷 (详见 tests/eval.py 与 docs/TEST_REPORT.md、docs/BENCHMARK.md)。局限: CPU 上交叉编码器精排较慢, 候选窗口 (默认 30) 决定精度上限; 融合权重与候选数为经验值, 可用评测脚本微调。

技术点 ②: 长期记忆 (mem0 范式: 抽取 → 去重/冲突更新 → 注入)

问题背景: 普通聊天系统每次会话“失忆”, 同一用户换个会话就得重新交代背景。要让系统“记得住你”, 需要在每轮对话后增量沉淀关于用户的稳定事实, 并在回答时按需取回。

算法思路:

输入: 本轮 (用户消息, 助手回答)、user_id

输出: 新记忆是否入库 (bool)

1. 抽取: LLM 从对话中抽取结构化事实 {type: identity|preference, subject: 属性名, content: 事实陈述} (无内容则输出 [])
2. 入库去重: 事实向量化 → 与库中该用户最近邻比较
 - 相似度 ≥ 0.85 → 视为重复, 跳过
 - 同 subject 且同 type 且相似度 ≥ 0.6 → 视为冲突, 删除旧事实, 写入新事实
 - 否则直接写入
3. 注入 (提问时): 把问题向量化检索该用户相关记忆 → 格式化为提示片段
 - 拼进系统提示 ("关于当前用户的长期记忆...")
 - 并拼接在查询前再做检索 (记忆增强检索), 提升个性化问题命中

关键代码 (src/memodoc/memory/store.py):

```
def add(self, fact, user_id="default"):
    content = fact["content"]
    emb = self.embedder.embed([content])
    if emb is None:
        # 无 embedding 的降级: 精确去重
        if any(it["text"] == content for it in self._store.all({"user_id":
user_id})):
            return False
        self._insert(fact, user_id, None)
        return True
    res = self._store.query(emb[0], 1, {"user_id": user_id})
    if res:
        sim = 1.0 - res[0]["distance"]
        meta = res[0]["meta"]
        if sim >= settings.memory_sim_threshold:
            return False
            # 高度相似: 去重
        if (meta.get("subject") == fact["subject"]
            and meta.get("type") == fact["type"] and sim >= 0.6):
            self._store.delete_ids([res[0]["id"]])
            # 同属性冲突: 替换
        self._insert(fact, user_id, emb[0])
        return True
```

效果 / 说明：演示中“我是大一新生，还需要交会费吗”这类问题因记忆增强检索可命中“入社条件”块，实现个性化回答与精确引用双赢。局限：记忆质量依赖 embedding 相似度与抽取 prompt；事实冲突的判定粒度是“同属性”，复杂人格演变（同属性多值）会简单替换。

技术点 ③：引用可信闭环（编号引用 + 引用核查）

问题背景：RAG 回答最容易受质疑的就是“来源是真是假”。若让模型自由写来源，可能编造。需要从机制上让每个 [n] 都对应真实片段，并事后核查。

算法思路：

生成阶段：

1. 检索片段按 1..k 编号写进系统提示，并强约束：只能引用提示中存在的编号
2. 片段文本与编号一一对应，回答里出现的 [n] 即第 n 个片段（机制上杜绝“乱标”）

核查阶段：

3. 收集回答中出现过的编号 n
4. 取对应片段，构造核查 prompt（你是引用核查器.....只输出 JSON）
5. LLM 判定每处引用 supported / unsupported，无法判定记 unknown

前端展示：

6. [n] 渲染为可点击 chip → 调 /api/open-file 打开源文件
7. 引用面板显示 ✓已核查 / △不支持 徽标；存在未通过项时气泡内给出汇总提示

关键代码 (src/memodoc/pipeline.py 的 check_citations 核心)：

```
cited = sorted({int(m) for m in _CITE_RE.findall(answer)})
if not cited:
```

```

    return {}
by_index = {r.index: r for r in retrieved}
items = [(n, by_index[n]) for n in cited if n in by_index]
if not items:
    return {}
system = ("你是引用核查器。判断回答中每个 [n] 引用的内容是否确实由对应的文档片段支持。
\n"
         "规则：片段中没有的信息，该引用必须标为 unsupported；片段明确支持则标
supported。 \n"
         "只输出 JSON 对象，形如 {\"1\": \"supported\", \"2\": \"unsupported\"}，不
要任何解释。")
body = ["回答：", answer, "", "文档片段："]
for n, r in items:
    body.append(f"[{n}] {r.text}")
data = self.generator.llm.chat_json(
    [{"role": "system", "content": system},
     {"role": "user", "content": "\n".join(body)}])

```

效果 / 说明：实测回答常出现“有引用但片段不支持”被核查揪出并打 △ 的情况（`tests/eval.py` 统计引用核查通过率，演示文档集基线 100%）。局限：核查本身多一次 LLM 调用（延迟与成本）；**unknown** 情形（如编号不存在）需要前端展示兜底。

技术点 ④：SSE 流式问答全栈管线

问题背景：长回答若等全部生成完再返回，等待感强；且引用来源、逐字内容、核查结果三类信息到达时机不同，需要一种协议边生成边推送。

算法思路（事件协议）：

```

前端 POST /api/chat {question, tags, use_memory}
后端以 text/event-stream 依次推送 (data: {...}\n\n) :
1. {type:"sources", items:[{index, doc_name, section, source, preview}...]}
   — 检索完成后先给来源，前端立即展示右栏
2. {type:"delta", text:...} 若干帧 — 逐字增量，前端追加渲染
3. {type:"checks", items:[{index, status}...]} — 引用核查结果
4. {type:"done", session_id} 或 {type:"error", message}
前端用 fetch + ReadableStream 逐帧解析 SSE；发送期间锁定输入，结束后刷新会话列表。

```

关键代码（`api/server.py` 的 SSE 生成器骨架）：

```

def gen():
    def emit(obj):
        yield f"data: {json.dumps(obj, ensure_ascii=False)}\n\n"
    try:
        full, retrieved = "", []
        for delta, chunks in pipe.answer_stream(sid, req.question,
                                                use_memory=req.use_memory,
                                                user_id=username, tags=req.tags or
None):

```

```
        if not retrieved:
            yield from emit({"type": "sources",
                            "items": [{ "index": r.index, "doc_name":
r.doc_name,
                                     "section": r.section, "source":
r.source,
                                     "preview": r.text.replace("\n", " ")
                                     }
                                     ]})
            for r in chunks:
                retrieved = chunks
                full += delta
                yield from emit({"type": "delta", "text": delta})
                checks = pipe.check_citations(full, retrieved) if retrieved else {}
                yield from emit({"type": "checks",
                                "items": [{ "index": n, "status": s} for n, s in
checks.items()]})
                yield from emit({"type": "done", "session_id": sid})
            except Exception as e:
                yield from emit({"type": "error", "message": str(e)})
                yield from emit({"type": "done"})
```

效果 / 说明：首字延迟显著缩短、来源先于正文到达、“生成—溯源—核查”三类信息各自成帧，前端状态机清晰（`store.ts` 的 `onEvent` 按 type 分发）。局限：SSE 需要独占连接，代理/网关需关闭缓冲；断线重连策略未实现，网络中断会以错误帧提示。

5.3 技术难点与解决方案

#	难点	解决方案
1	本机无 MSVC 编译工具，ChromaDB 依赖的 chroma-hnswlib 无 Windows/3.12 预编译 wheel，无法安装向量库	自研轻量向量库：numpy 精确余弦检索 + JSON 持久化；演示规模（数百块）下毫秒级完成，零原生依赖、可审计；接口对齐 ChromaDB 风格，后续可无缝替换正式库
2	本地 embedding / 重排模型体积大、下载易中断，且无代理时 HuggingFace 不可达	<code>model.py</code> 实现 Range 断点续传 + 自动重试； <code>.env</code> 支持切换 <code>hf_endpoint</code> 镜像；加载失败自动降级（纯向量 / 纯 BM25 / 精确去重），保证无模型也能演示
3	"中文问英文论文"检索不到：BM25 中英文分词互相干扰，向量对专名不敏感	BM25 按文档语言自适应分词（中文 jieba / 英文按词切分）；中文查询先经 LLM 译成英文，双语各自"向量+BM25"融合后按 id 合并，再由多语言重排器精排；分词器带版本号，变更自动重建索引
4	回答来源难以让人信服，存在幻觉风险	机制 + 核查双保险：编号引用与片段一一对应写入提示（禁止编号）；生成后 <code>check_citations</code> 逐条 LLM 核查；前端把 <code>[n]</code> 做成可点击 chip 打开源文件，并显示 ✓/⚠ 徽标
5	首字符延迟与 CPU 推理开销	模型全部本地化 + 流式输出；检索候选先融合后精排、只对 top_k 重排控制开销；embedding 批量编码；生成前先发 sources 帧让用户立刻看到进度

六、部署与使用说明

6.1 环境要求

项目	要求
操作系统	Windows 10/11（本机开发与答辩演示环境）
运行时	Python 3.11–3.12（推荐 3.12，项目锁定 ❤️.13）；Node.js 18+（仅前端构建需要）
依赖库	后端见 pyproject.toml / uv.lock （fastapi、uvicorn、pydantic、pydantic-settings、openai、pymupdf、jieba、numpy、sentence-transformers 等）；前端见 frontend/package.json
其他	需联网调用 DeepSeek API（在 .env 填 DEEPSEEK_API_KEY ）；首次需联网下载本地模型（约 2.4GB，可用镜像/断点续传）；本机已含模型可直接离线演示

6.2 安装与部署步骤

- 安装 [Python 3.12](#) 与 [uv](#)（也可跳过 [uv](#)，用 `python -m venv + pip`）。
- 获取代码（解压提交包或 `git clone` 仓库），进入项目根目录。
- 创建虚拟环境并安装后端依赖：

```
uv sync                                # 或：python -m venv .venv && .venv\Scripts\pip
install -e .
```

- 配置密钥：

```
copy .env.example .env # 编辑 .env，填入 DEEPSEEK_API_KEY=sk-...
```

- 下载本地模型（可选，若未随包提供；支持断点续传）：

```
.venv\Scripts\python.exe -m memodoc.cli download-model
.venv\Scripts\python.exe -m memodoc.cli download-model --rerank
```

- 构建前端（首次）：

```
cd frontend
npm install
npm run build
cd ..
```

- 启动系统：

```
.venv\Scripts\python.exe server.py
```

浏览器打开 <http://127.0.0.1:8000>。已构建过一次后，可直接双击根目录 `start.bat` 一键启动。

开发模式（前端热更新）：另开终端 `cd frontend && npm run dev`，访问 <http://127.0.0.1:5173>（/api 自动代理到 8000）。首次启动若账号库为空会自动创建默认管理员 `admin / admin123`；如需干净演示数据，可删除 `data/store/users.json`、`data/store/tokens.json` 与 `data/sessions/` 后重启。

6.3 使用说明

1. 打开 <http://127.0.0.1:8000>，用 `admin/admin123` 登录，或注册一个普通账号（user 角色）。
2. **建立知识库**：进入「📁 文件库」Tab → 选择文件（PDF/MD/TXT，可多选）→ 可选填写标签 → 上传。系统自动解析分块、向量化、建立索引；未填标签时会自动打标签。上传完成后文档出现在列表中，可随时增删标签、下载、删除（非本人且非管理员不能删）。
3. **提问**：切到「💬 问答」Tab，输入问题（或点建议问题，如“状态图怎么画？”）。回答流式逐字显示，右上引用面板同步展示本次用到的来源片段。
4. **溯源与核查**：回答中的 `[n]` 是可点击的引用 chip，点击即用系统默认程序打开源文件；来源卡片带「✓ 已核查 / △不支持」徽标，若某处引用未通过核查，气泡下方会给出提示。
5. **限定检索区间**：问答输入框上方可多选标签（如只检索“课件”），提问将只在带这些标签的文档内检索。
6. **会话管理**：左侧栏可新建 / 切换 / 删除会话，历史自动保存。
7. **长期记忆**：无需手动操作——回答结束后系统自动抽取关于你的身份/偏好记忆，下次（哪怕新会话）提问会自动注入相关记忆。可用命令行查看/清空：

```
.venv\Scripts\python.exe -m memodoc.cli memories
```

8. **管理员**：可进入「👤 用户管理」查看全部注册用户。

6.4 项目目录结构

```
handfree/ (MemoDoc 提交根目录)
├── server.py           # 启动入口: uvicorn 拉起 FastAPI (8000)
├── start.bat          # 一键启动 (Windows)
├── pyproject.toml / uv.lock # 后端依赖与锁定文件
├── .env.example       # 配置模板 (复制为 .env 并填 key)
├── README.md         # 项目说明 (运行/使用/结构)
├── 软件开发文档.md   # 本文档
├── api/              # FastAPI Web 层
│   └── server.py     # 全部路由: 认证/会话/SSE 聊天/文档库/标签/上传/下载/打开
源文件
├── auth.py           # 账号系统 (user/admin、加盐哈希、令牌)
├── services.py       # 文档所有权元数据
├── src/memodoc/      # 核心: RAG + Memory (冻结逻辑)
│   ├── config.py     # 配置: llm/openai_compat.py LLM 统一客户端
│   └── rag/           #
parser/chunker/embedder/store/sparse/reranker/retriever/generator
```

```

|   |— memory/                # extractor/store/injector
|   |— session.py / tagger.py / pipeline.py / model.py / cli.py
|— frontend/                  # React + TS 前端（构建产物由 FastAPI 托管）
|   |— package.json / vite.config.ts / tsconfig*.json
|   |— src/                   # types/api/store/App/styles.css + components/（7 个
组件）
|— tests/                     # smoke / 接口用例(28 个) / eval / eval_papers /
eval_ragas
|— docs/                      # 有效设计文档：ARCHITECTURE / DESIGN_SPEC /
TEST_REPORT / BENCHMARK
|— data/                      # 运行时数据（uploads 源文件、store 向量/记忆/账号、
sessions 历史、models 模型）

```

七、项目总结

7.1 收获与反思

技术上的最大收获，是把“RAG 聊天”从黑盒调用变成了亲手实现的完整链路：手写 BM25（分词、倒排、打分）、自研 numpy 向量库、双路归一化融合、交叉编码器重排、引用核查、记忆抽取/去重/注入——每一步都对应一个能解释清楚的设计决策（比如为什么不用 ChromaDB、为什么给分词器加版本号、为什么记忆存结构化事实而非原文）。这些决策记在 `docs/ARCHITECTURE.md` 里，答辩时“为什么这么做”都有据可答。

工程上的体会：一是**分层与解耦的价值**——核心 `src/` 冻结不动，Web 层（FastAPI）与产品层（文档所有权）只做组合与鉴权，让我在多次前端改版时不需要动检索逻辑；二是**测试与文档先行**——28 个接口用例与评测脚本让“改完还敢上线”，测试报告与架构文档在答辩前就是可信的证据；三是**工具链**：uv 管理依赖、单端口托管前端产物、JSON 落盘可审计，这些小选择大幅降低了演示环境的复杂度。

不足与反思：产品规划偏大（早期设想的师生双角色、学习画像、班级统计在 React 版中因时间被收敛为 user/admin + 问答/文件库/用户管理），说明功能取舍要更早、更果断；记忆与画像等能力的 UI 展示没有全部落地；代码行数增长后，模块内部分函数缺少单测，主要靠接口用例与人工演示覆盖。另外个人时间管理上，前期调研投入较多，留给打磨与联调的时间偏紧——文档里“先跑测试再演示”的习惯帮我兜住了底线。

7.2 未来改进方向

- 将设计文档中已定义的**学习画像与班级薄弱点统计**（`learning` 事实类型 + 聚合接口）重新接入 React 前端，形成师生双角色闭环。
- 前端类型改为由 FastAPI OpenAPI 自动生成（openapi-typescript），消除手写类型漂移；补充账号改密、找回等基础能力。
- 向量库替换为正式方案（ChromaDB/Qdrant）或按需引入分片与增量索引，支撑更大语料；检索参数（融合权重/候选数）支持配置化微调。
- 补齐 RAGAS / RGB 等外部基准评测并自动化（`docs/BENCHMARK.md` 已给出接入路径），让指标可复现、可对外陈述。
- 容器化与一键部署脚本（Dockerfile + 启动编排），便于在教室/机房环境快速拉起演示。